Learning to generate

Autoencoders, VAEs and GANs, built from one handwritten 3



A 3 turns into a 7. Every frame is decoded from a point on a straight line through the code space of the VAE we train today.

Nipun Batra · IIT Gandhinagar

START WITH A QUESTION

Who are these people?



No one. Each face is a random vector of 512 numbers pushed through a trained network.

Karras, Laine & Aila, *A Style-Based Generator Architecture for GANs* (StyleGAN), 2019 · CC BY-NC 4.0

YOUR TURN TO BE THE JUDGE

Real or generated?



Six are MNIST test digits, six come from a network. Write down which six are fake.

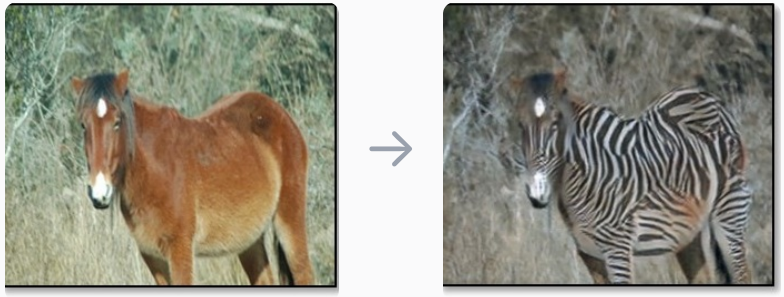
GENERATION IS NOT ONLY "MAKE A RANDOM PICTURE"

Predict the output, then look

a sketch → a photo



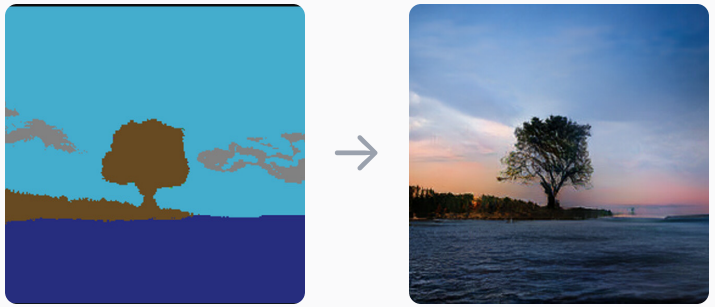
a horse → a zebra



fill a hole



a label map → a landscape



> Each input allows many plausible outputs. A generator must pick one that looks real.

pix2pix (Isola et al. 2017) · CycleGAN (Zhu et al. 2017) · Context Encoders (Pathak et al. 2016) · SPADE (Park et al. 2019). All published model outputs.

ONE INPUT, MANY ANSWERS

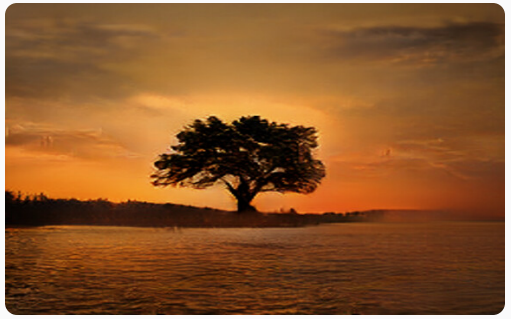
The same label map, three different photographs



the condition



sample 1



sample 2



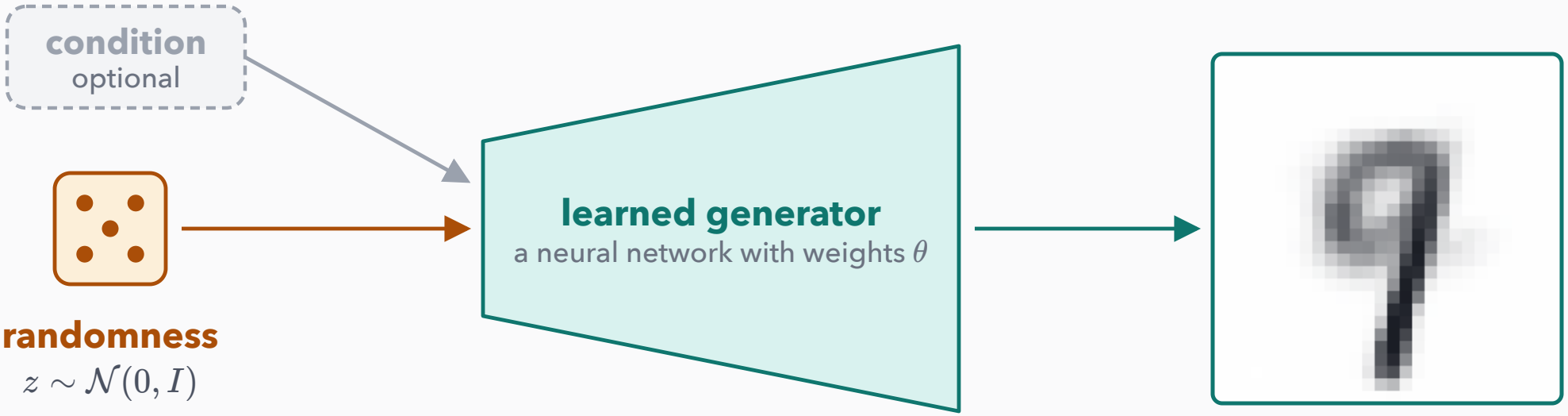
sample 3

A generator does not learn one answer. It learns a distribution of answers and draws from it.

SPADE / GauGAN, Park et al. 2019, teaser figure: here the variation comes from a style image given to the generator.

WHAT ALL OF THESE SHARE

Randomness in, a learned network, a sample out



e.g. a sketch, a label, a sentence

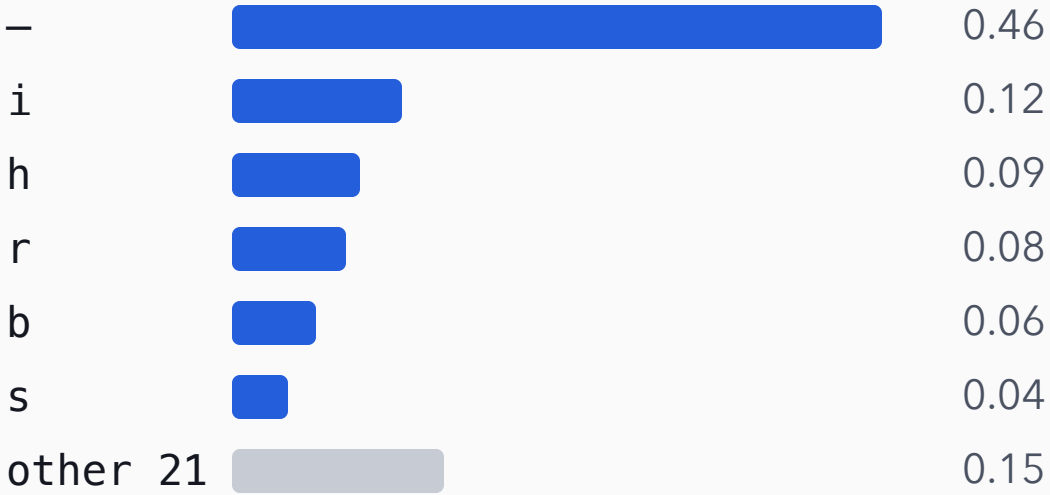
Training means: adjust θ until the samples look like the data. But what is the loss?

PART 1 OF THIS COURSE

You have already built a generator

Our name model reads three characters and gives a probability for every next character.

after `a a b`, the trained model says:



Generate: sample a character, append it, repeat.



A different random draw at any step gives a different name. That is a generative model: it gives probabilities, and we sample from them.

Same idea today: learn a distribution, then sample from it. Only now the samples are images.

PART 1 OF THIS COURSE

A whole name gets one probability

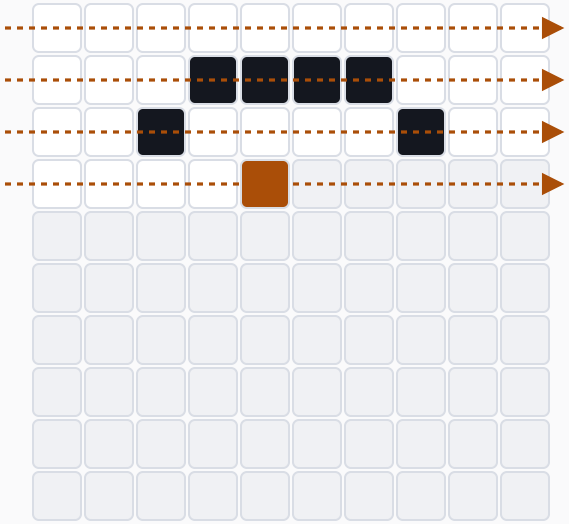
context	next	probability	−log p
– – –	a	0.08	2.53
– – a	a	0.08	2.53
– a a	b	0.02	3.91
a a b	i	0.12	2.12
a b i	d	0.09	2.41
b i d	–	0.23	1.47
aabid		**3.2×10^{-7}**	**14.96**

$$p_\theta(\text{aabid}) = \prod_t p_\theta(x_t \mid x_{<t}) \qquad -\log p_\theta(\text{aabid}) = \sum_t -\log p_\theta(x_t \mid x_{<t})$$

> **Training made this number large for every name in the data. That is maximum likelihood, and it is what we want for images too.**

Two ways to generate a 28 × 28 digit

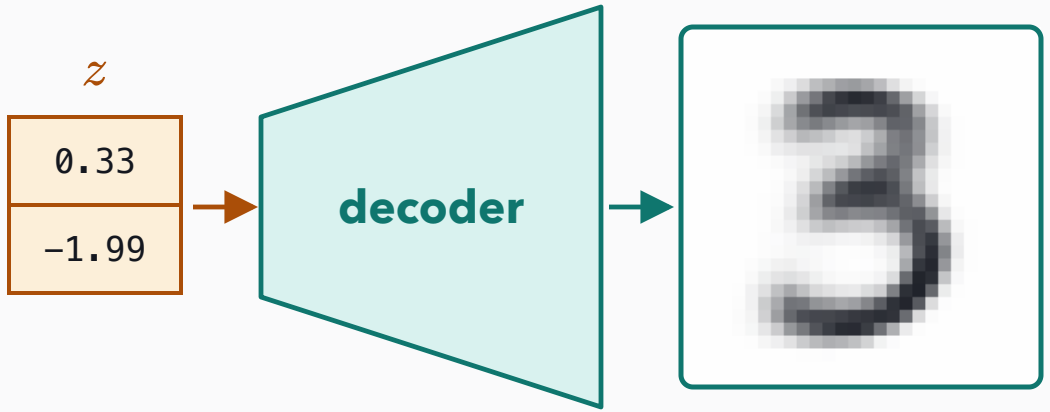
A · one pixel at a time



784 predictions in a row, like our names

$$p(x) = \prod_i p(x_i \mid x_{<i})$$

B · all pixels from one random vector



one pass: 2 numbers in, 784 pixels out

$$x = \text{decoder}(z)$$

Today: route B. Two questions: where does z come from, and how do we train a decoder when no one tells us the right z?

THE PLAN

Today's route

1. **Distributions and hidden causes** on a 1-D toy dataset
2. **Autoencoders** and why they cannot generate
3. **KL divergence** how far apart are two distributions?
4. **The ELBO** a loss for a model we cannot evaluate exactly
5. **Reparameterisation** backprop through a random draw
6. **The VAE on MNIST** one digit, end to end, with real numbers
7. **GANs** learn the judge instead of the likelihood

> **One running image throughout: test digit #90, a handwritten 3.**

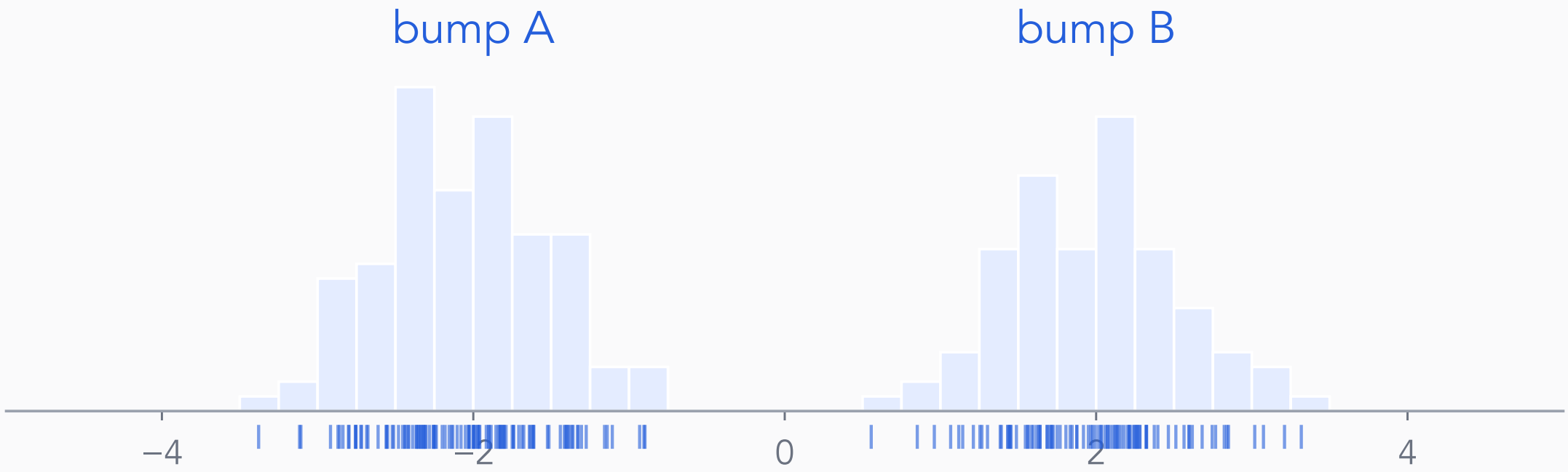
PART 1

What does it mean to learn a distribution?

Before images, one number. Everything we need (fitting, sampling, hidden causes, inference) fits on a number line.

OUR 1-D TOY DATA

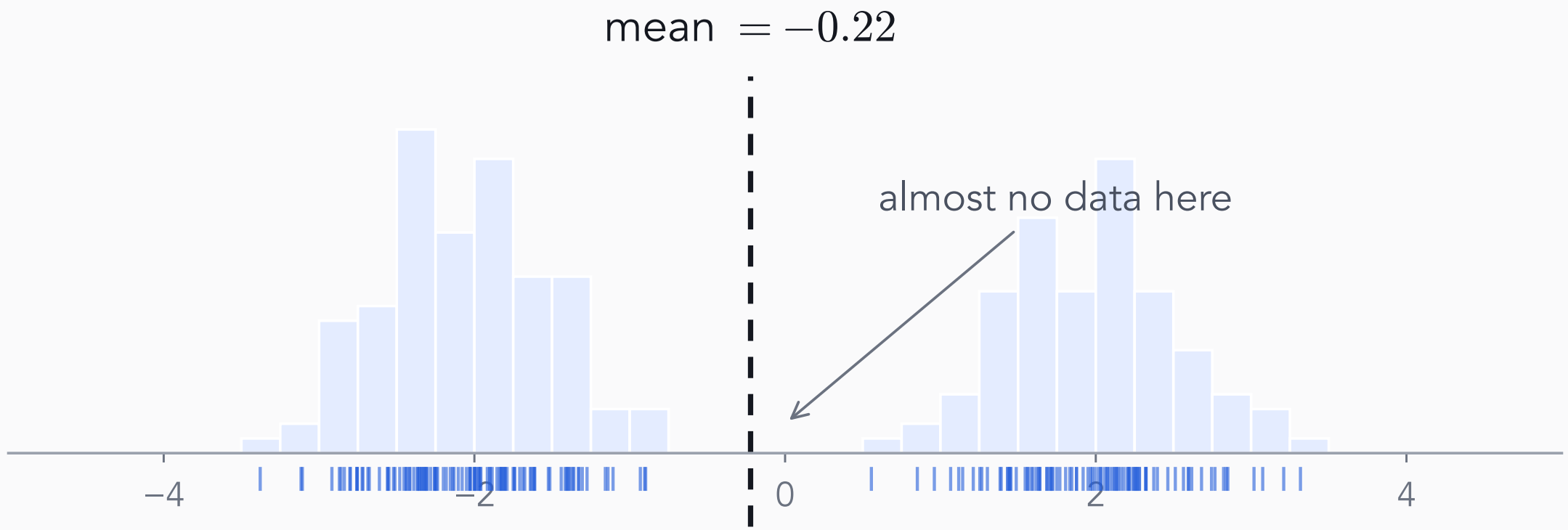
200 numbers in two clumps



> **Pretend each number is an image squashed to one value. How would you produce a 201st that looks like it belongs?**

A TEMPTING ANSWER

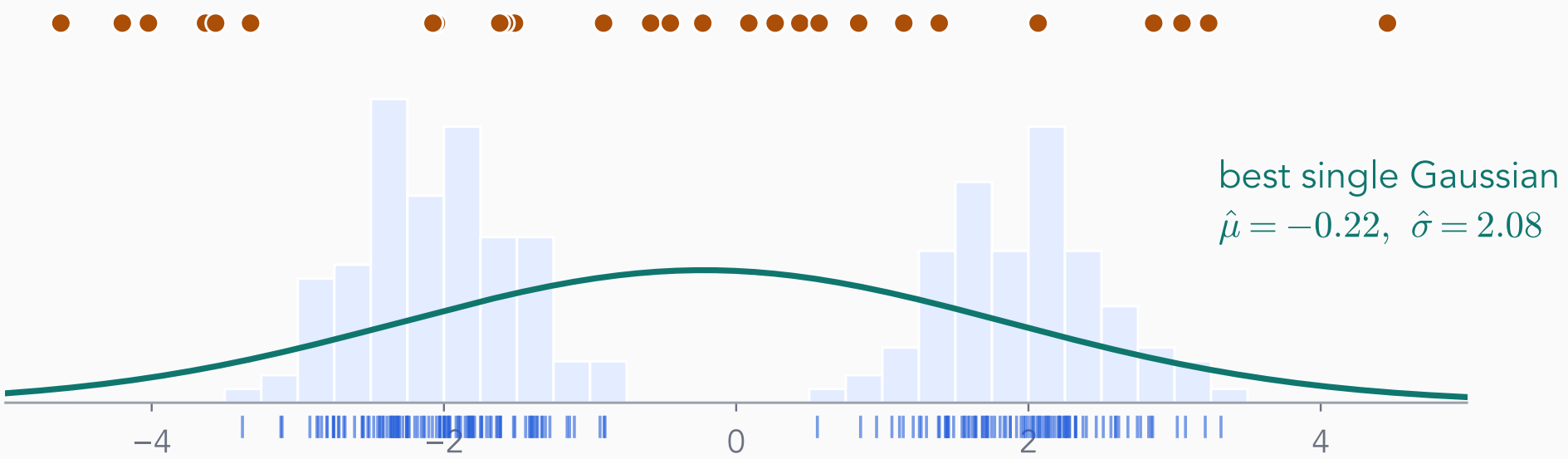
The average is a terrible sample



A model that predicts the mean always outputs something that never occurs. Regression thinking does not give generation.

FIT, THEN SAMPLE

The best single Gaussian, by maximum likelihood

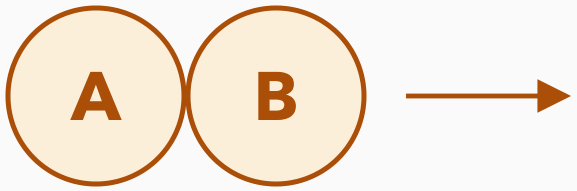


$$\hat{\mu} = \frac{1}{N}\sum_{i=1}^{N} x_i = -0.22 \qquad \hat{\sigma}^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{\mu})^2 = 4.31 \qquad x_{\text{new}} = \hat{\mu} + \hat{\sigma}\,\varepsilon, \quad \varepsilon \sim \mathcal{N}(0,1)$$

11 of 30 samples land in the empty gap. Best fit within the family, wrong shape.

Hidden choice first, then the number

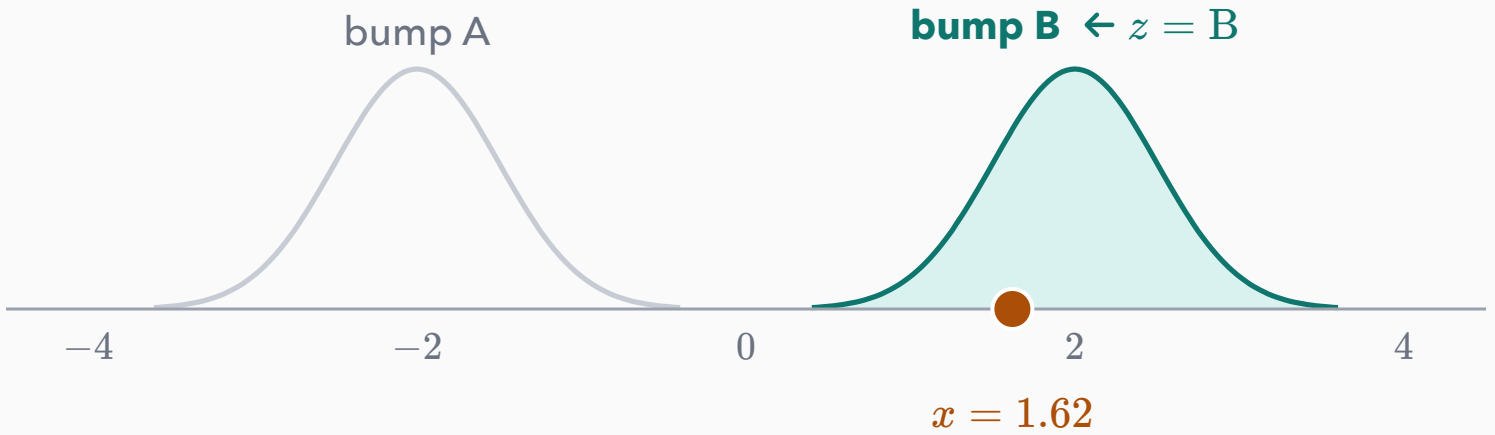
step 1 · hidden choice



flip a fair coin:

$p(z{=}\mathrm{A}) = p(z{=}\mathrm{B}) = \frac{1}{2}$

step 2 · draw x from the chosen bump



> The coin is never written down in the data. We only ever see x. The coin z is a hidden (latent) variable.

ONE INGREDIENT, WORKED BY HAND

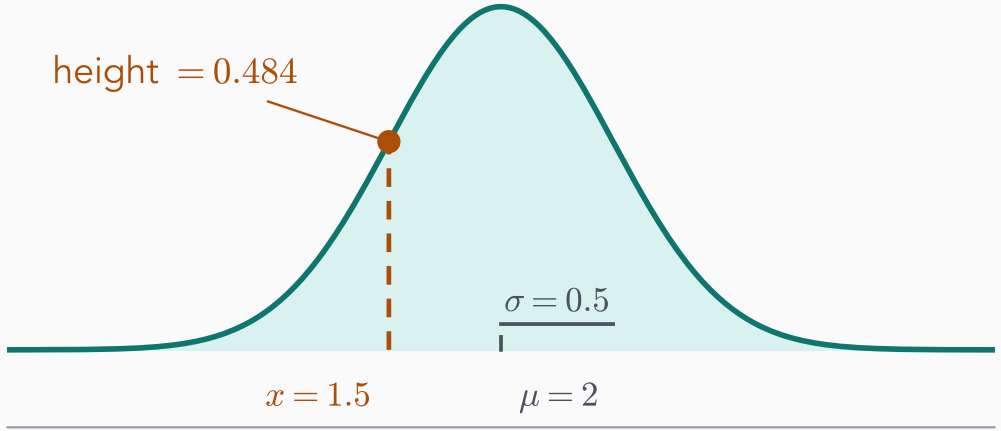
How high is bump B at $x = 1.5$?

$$\mathcal{N}(x;\mu,\sigma^2) = \frac{1}{\sigma\sqrt{2\pi}}\exp\Big(-\frac{(x-\mu)^2}{2\sigma^2}\Big)$$

$$\frac{x-\mu}{\sigma} = \frac{1.5-2}{0.5} = -1$$

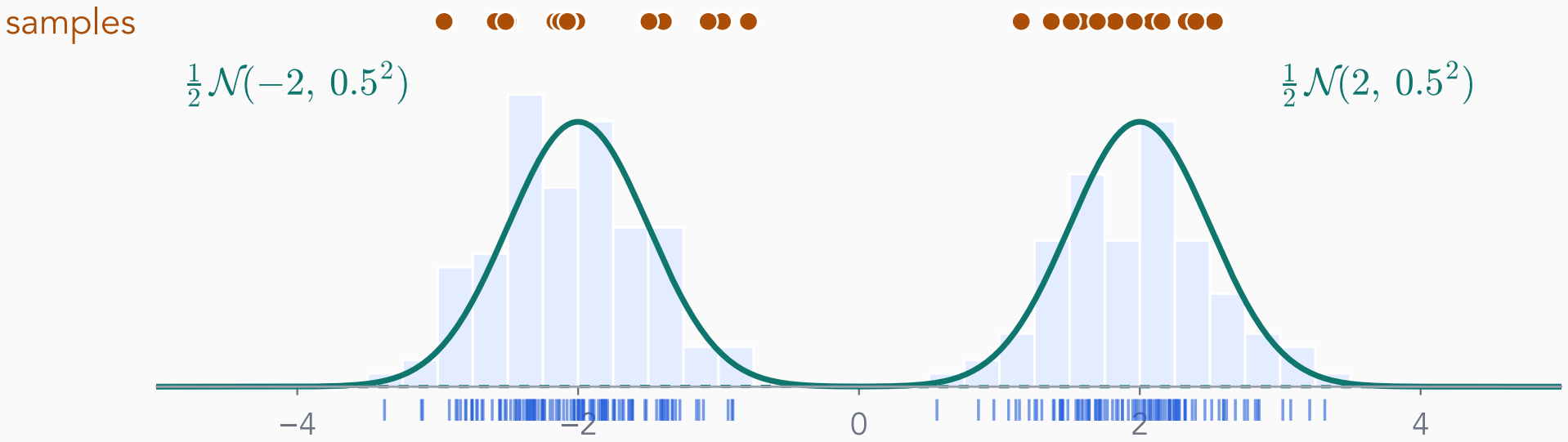
$$\exp\Big(-\tfrac{1}{2}\cdot(-1)^2\Big) = e^{-0.5} = 0.607$$

$$\frac{1}{0.5\sqrt{2\pi}} = 0.798 \qquad \Rightarrow \qquad 0.798 \times 0.607 = \mathbf{0.484}$$



> A density is a height, not a probability: it can exceed 1 for a narrow bump. Only areas are probabilities.

To score x, add over every hidden value



$$p(x) = \sum_{z \in \{\mathrm{A},\mathrm{B}\}} p(z)\, p(x \mid z) = \tfrac{1}{2}\,\mathcal{N}(x; -2, 0.5^2) + \tfrac{1}{2}\,\mathcal{N}(x;\ 2, 0.5^2)$$

$$\text{at } x = 1.5: \quad \tfrac{1}{2} \cdot 0.000 + \tfrac{1}{2} \cdot 0.484 = 0.242$$

> **Samples now fall in the bumps. Remember this sum. With images it becomes an integral we cannot compute.**

RUN THE STORY BACKWARDS

You see $x = 0.1$. Which bump made it?

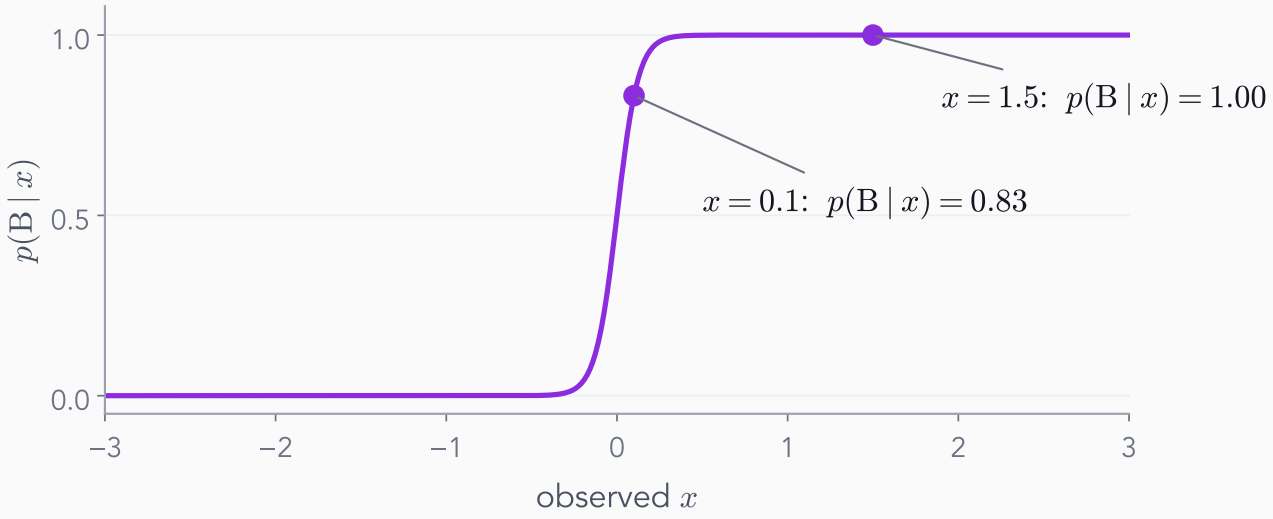
bump z	prior $p(z)$	likelihood $p(x \mid z)$	prior × likelihood	÷ total = posterior
A, centre -2	0.5	1.18×10^{-4}	5.89×10^{-5}	0.168
B, centre 2	0.5	5.84×10^{-4}	2.92×10^{-4}	**0.832**
total $= p(x)$			3.51×10^{-4}	**1.000**

$$p(\mathrm{B} \mid x) = \frac{p(\mathrm{B})\, p(x \mid \mathrm{B})}{p(x)} = 0.832$$

Bayes' rule is just this table: multiply, then divide by the total. $x = 0.1$ is close to the middle, so we stay unsure.

DO IT FOR EVERY x

The posterior, as a curve



Repeat the table for every x and plot $p(\mathrm{B} \mid x)$.

Far right: certainly B. Far left: certainly A. Near 0: unsure.

For these two bumps the curve is exactly $\mathrm{sigmoid}(16x)$: a logistic regression in disguise.

> **Going from an observation back to its hidden cause is called inference. The VAE's encoder will learn to do it.**

FOUR WORDS WE WILL USE ALL LECTURE

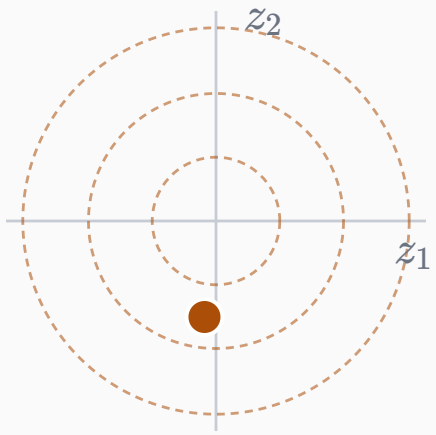
Prior, likelihood, evidence, posterior

name	symbol	question it answers	at $x = 0.1$
prior	$p(z)$	what do we believe about z before seeing x?	0.5, 0.5
likelihood	$p(x \mid z)$	if z were known, how probable is this x?	5.84×10^{-4} for B
evidence	$p(x)$	how probable is x overall, summing over z?	3.51×10^{-4}
posterior	$p(z \mid x)$	having seen x, which z explains it?	B: 0.832

Bayes ties them together: posterior = prior × likelihood ÷ evidence.

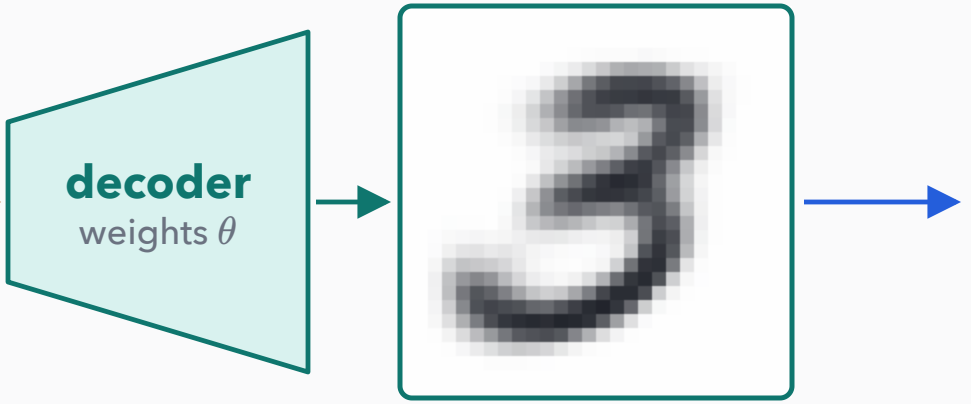
The same story for images

1 · draw a code



$$z \sim \mathcal{N}(0, I), \quad z = (-0.18,\ -1.50)$$

2 · decode probabilities



$$\pi = \text{sigmoid}(\text{decoder}(z))$$

3 · flip each pixel



$$x_i \sim \text{Bernoulli}(\pi_i)$$

> The code is 2 real numbers instead of a coin, and a network maps it to 784 pixel probabilities. Nobody tells it what z means.

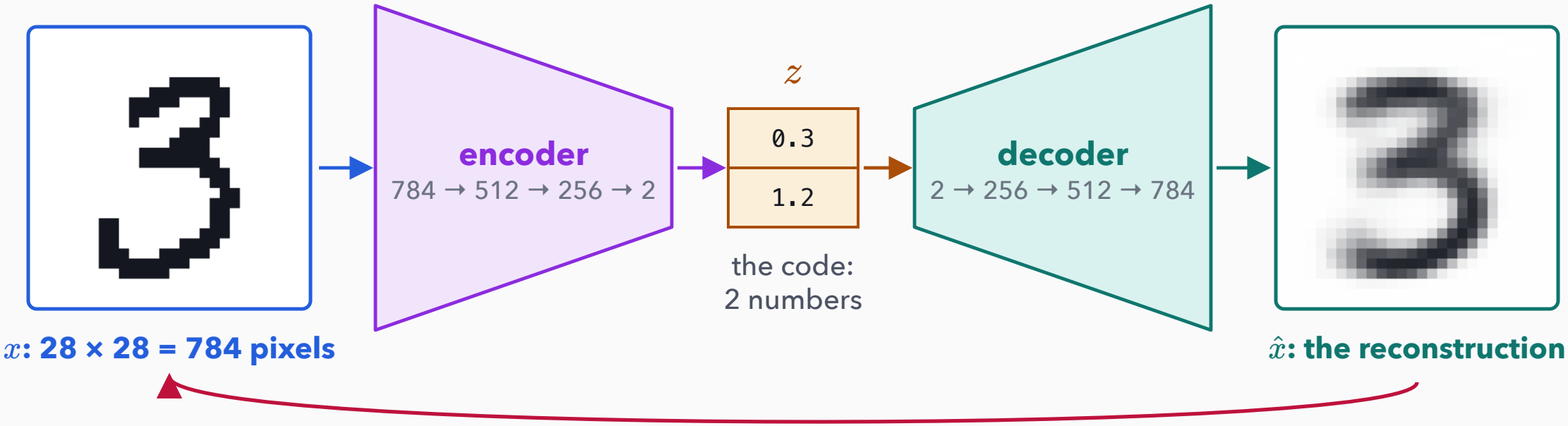
PART 2

First attempt: an autoencoder

Compress an image into a few numbers, then rebuild it. Then we try to generate with it, and watch that fail.

SQUEEZE, THEN REBUILD

An autoencoder on our 3



> No labels needed: the image is its own target. The 2 numbers in the middle are all the decoder gets.

COMPARING THE REBUILD WITH THE ORIGINAL

Binary cross-entropy, one pixel at a time

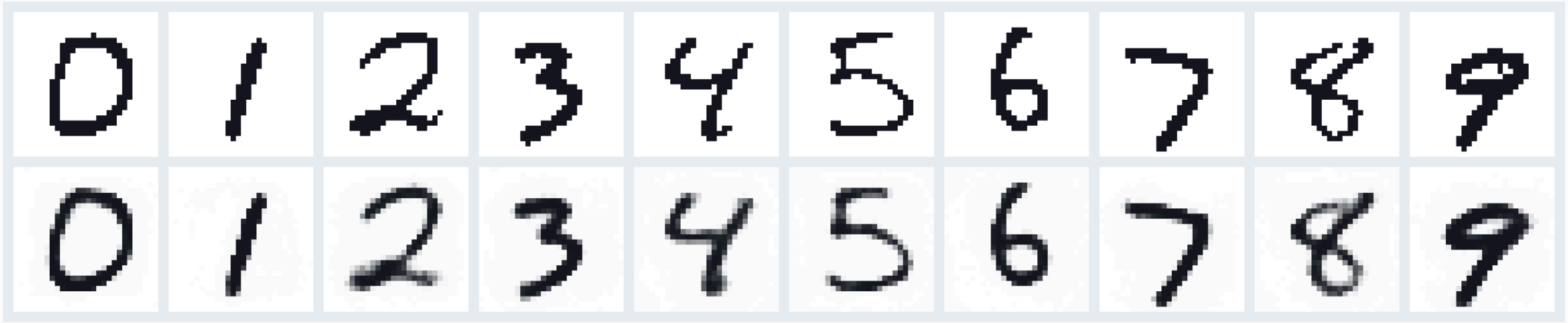
$$\text{loss}(x, \hat{x}) = -\sum_{i=1}^{784} \Big[x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i) \Big]$$

pixel	true value	predicted	loss
ink, predicted well	1	0.90	$-\log 0.90 = 0.11$
paper, predicted well	0	0.10	$-\log 0.90 = 0.11$
ink, missed	1	0.20	$-\log 0.20 = 1.61$

Pixels are 0 or 1 here (we binarise MNIST). Later we will see this is exactly a log-likelihood.

TRAINED ON 60,000 DIGITS

It reconstructs very well



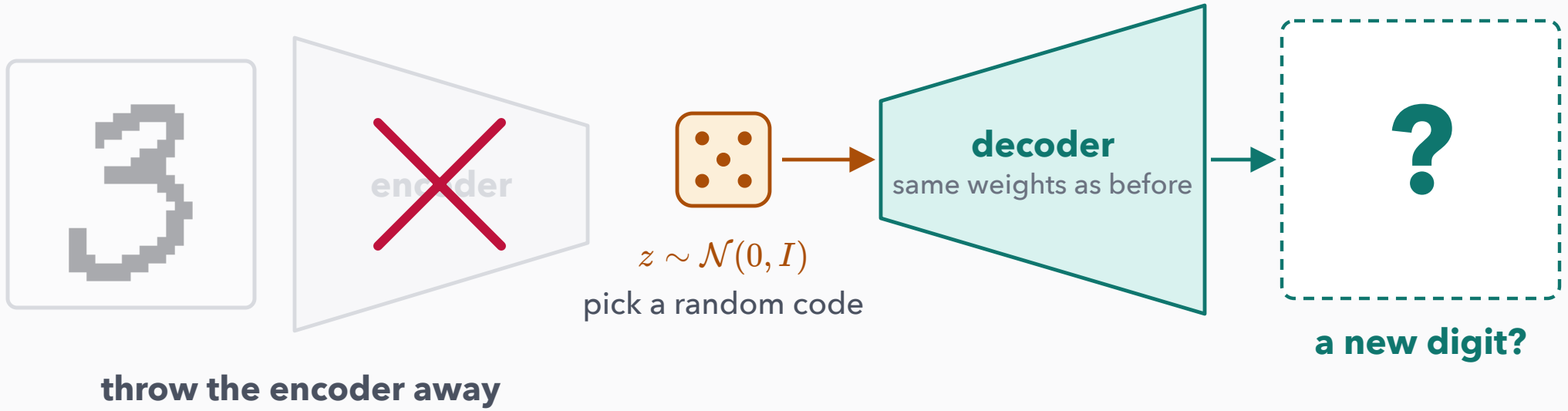
top: test digits it never saw

bottom: decoded from a 16-number code

784 pixels squeezed into 16 numbers and back. Average loss 44.9 nats per image. Can we now make new digits?

THE OBVIOUS IDEA

Keep the decoder, feed it a random code



The decoder turns codes into digits. So a random code should give a random digit. Right?

WHAT ACTUALLY COMES OUT

Random codes give scribbles

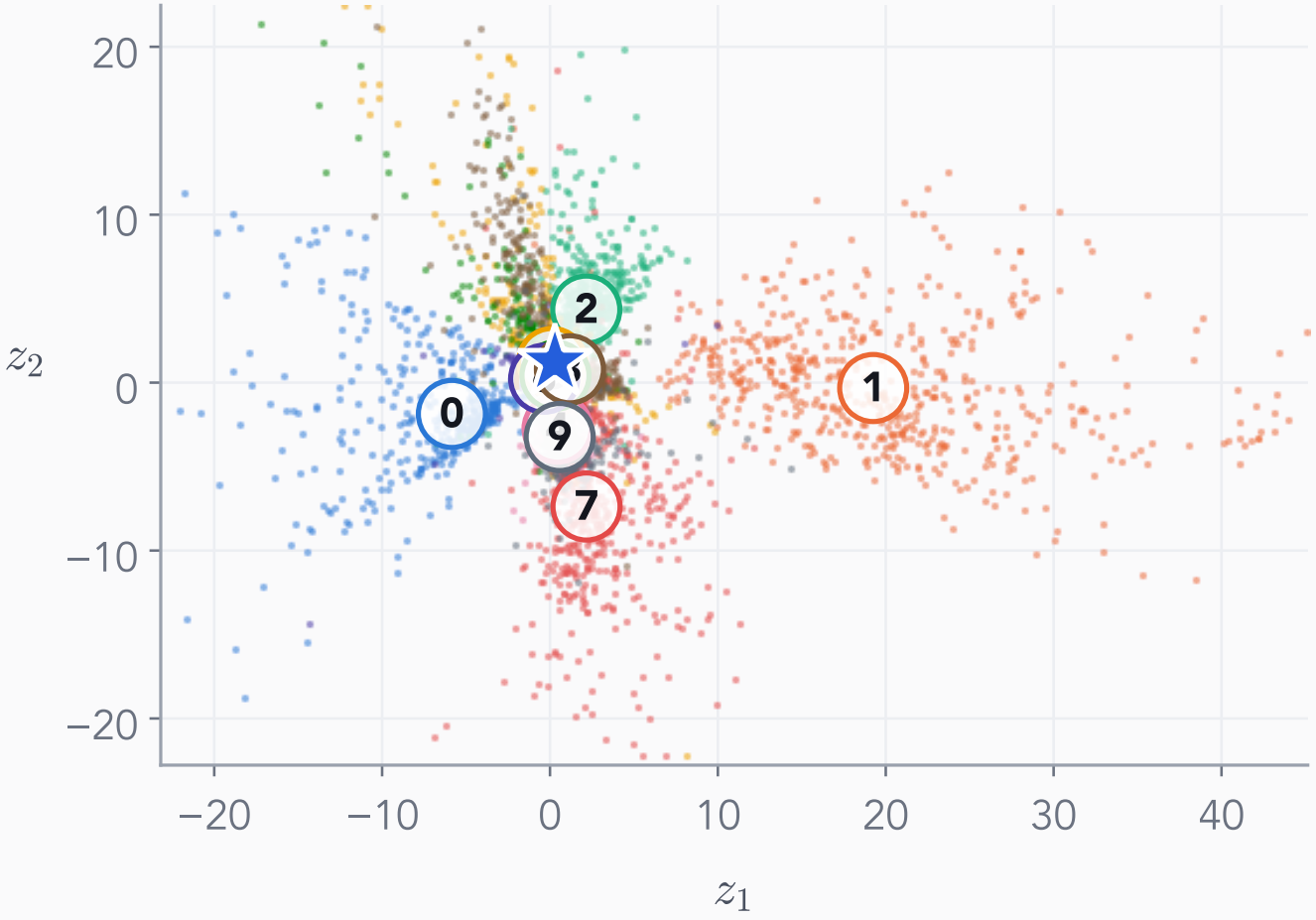


ten codes drawn from $\mathcal{N}(0, I)$ in 16 dimensions, through the trained decoder

The decoder only ever saw codes made by the encoder. Where do those codes live?

LOOK INSIDE (2-NUMBER VERSION)

Where 5,000 test digits land



Each dot is one test digit's code, coloured by its true label. The star is our 3.

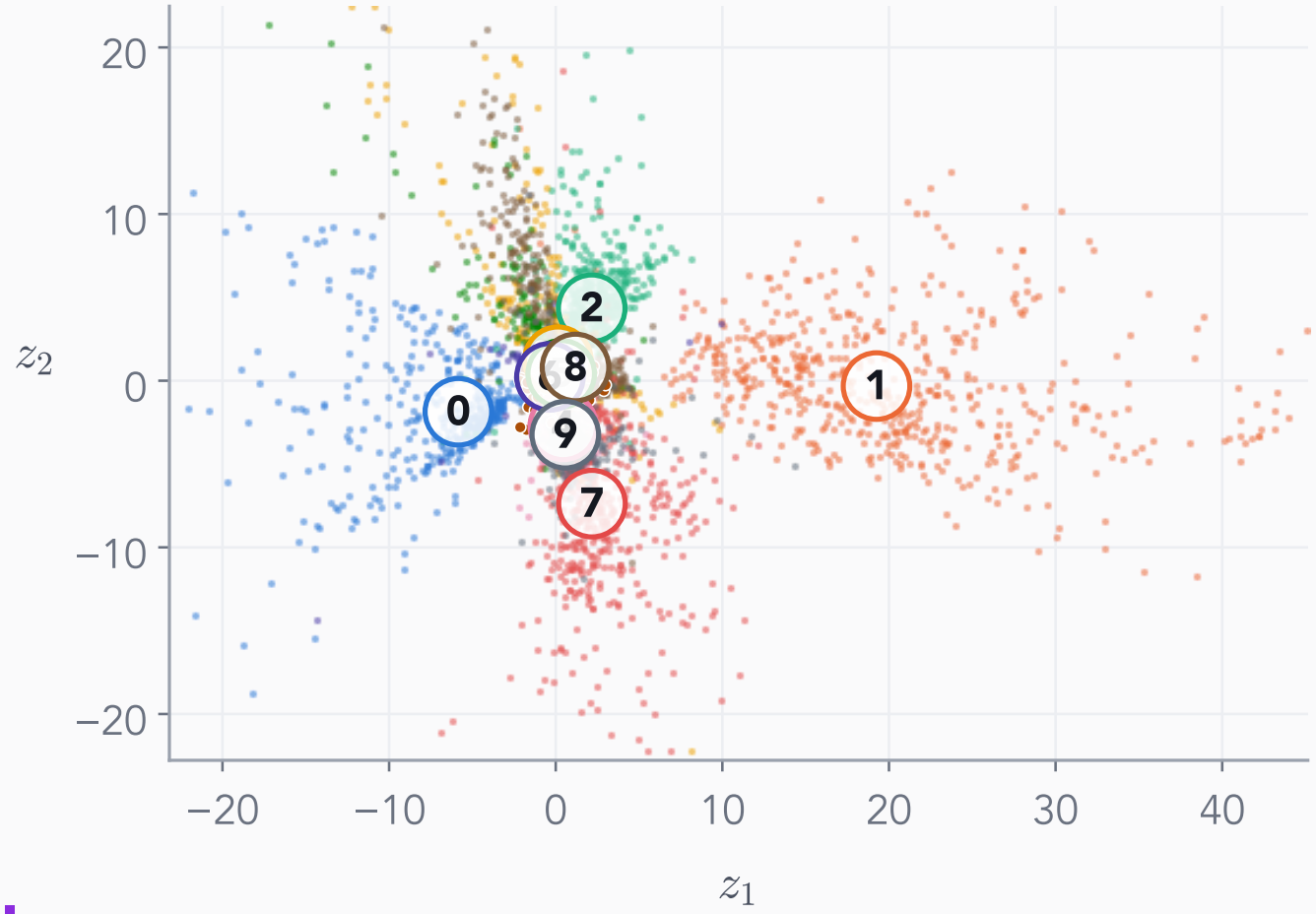
Same digits cluster together. Nobody asked for that; it just helps reconstruction.

Look at the axes: z_1 runs from −18.5 to 40.4. The dashed circles mark where $\mathcal{N}(0, I)$ puts its samples.

The code space has an arbitrary shape, scale and position.

OVERLAY RANDOM CODES

Random draws all land in one small spot



150 draws from $\mathcal{N}(0, I)$ in amber: all inside one tiny crowded patch.



ten of them decoded: some digits, some smudges

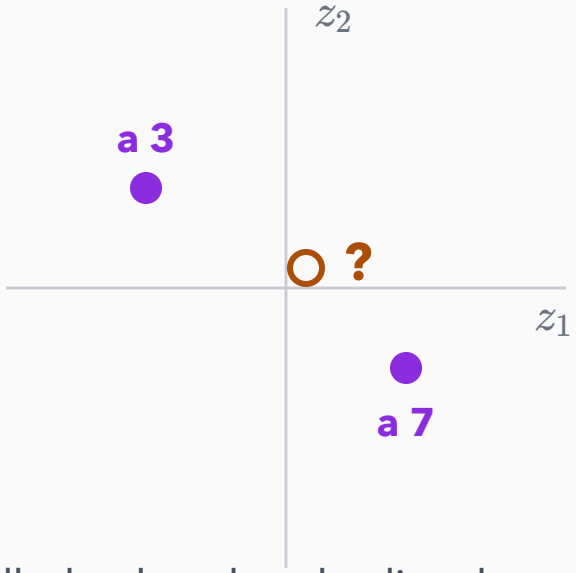
And we can never generate a 1: the 1s live near $z_1 \approx 20$.

To sample, we need to know where the codes are, and every point there must decode to something sensible.

WHAT WE WANT INSTEAD

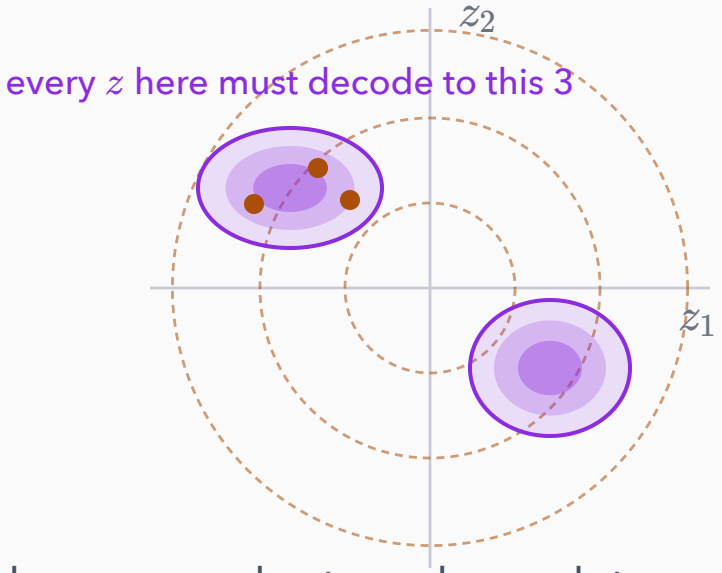
Encode to a cloud, not a point

autoencoder: a point



nothing tells the decoder what lives between codes

VAE: a cloud



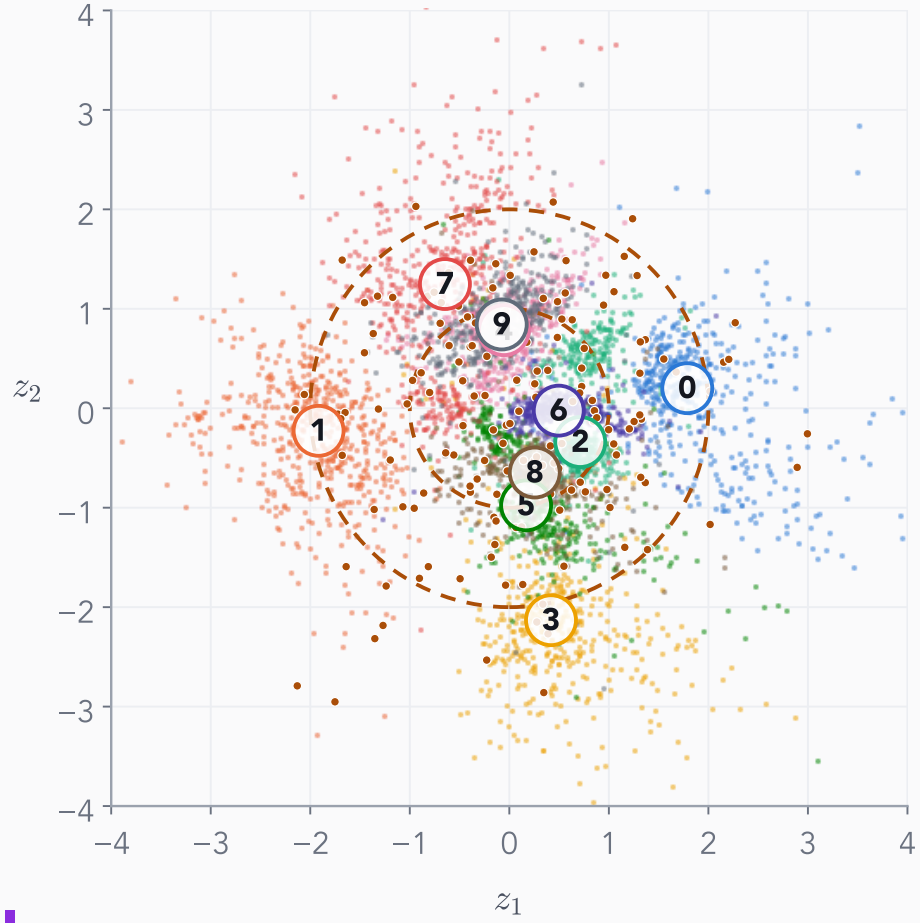
codes are spread out, overlap, and stay near $\mathcal{N}(0, I)$

Known: pull every cloud towards $\mathcal{N}(0, I)$, so random draws land on real codes.

Smooth: every point in a cloud must decode to that image, so neighbourhoods are meaningful.

A PREVIEW OF WHERE WE ARE GOING

The same plot for a VAE



Same data, same 2-number code, same random draws. Now the codes sit inside $\mathcal{N}(0, I)$.



the same kind of random codes, decoded

To get here we need two tools: a distance between distributions (KL), and a principled loss (the ELBO).

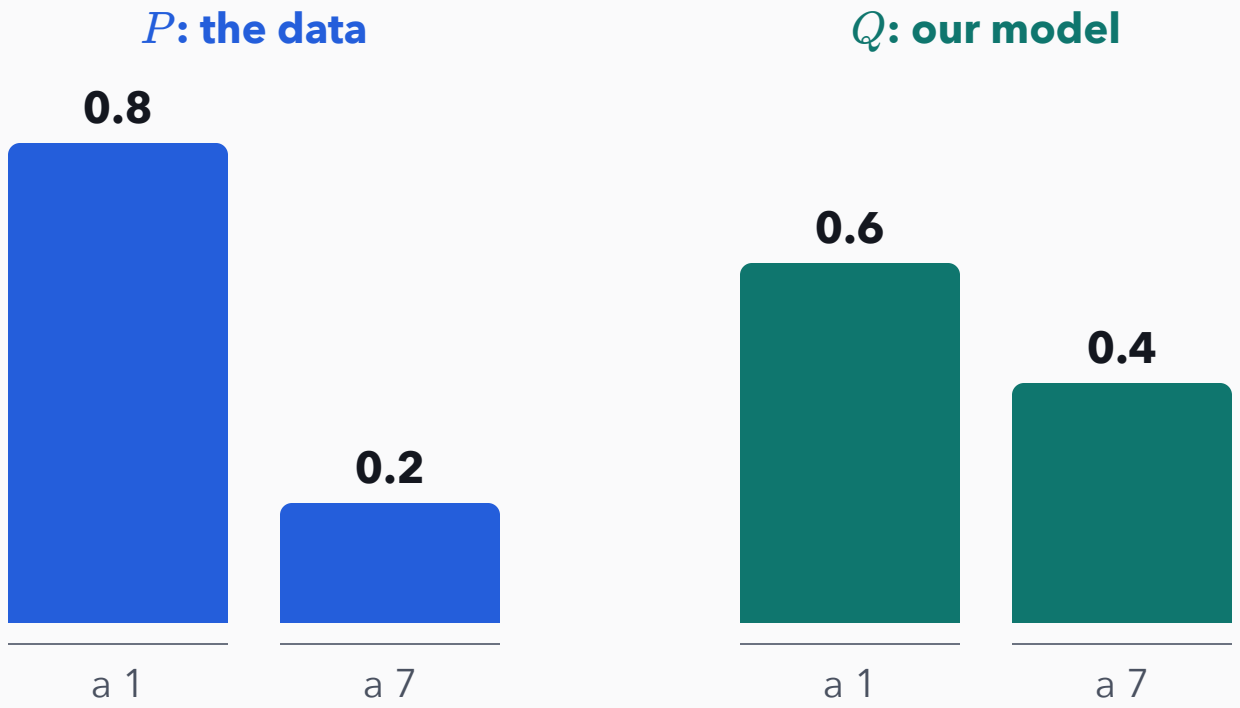
PART 3 · TOOL 1

KL divergence: how far is one distribution from another?

We need a number for "this cloud is far from $\mathcal{N}(0, I)$". Build it from surprise, with two bars first.

TWO DISTRIBUTIONS OVER THE SAME TWO OUTCOMES

The data says 80/20. The model says 60/40.



A dataset of 1s and 7s, 80% 1s. The model believes 60/40. How wrong is it, as one number?

INGREDIENT: SURPRISE

Surprise $= -\log(\text{probability})$

outcome	P	Q	surprise if you believe P	surprise if you believe Q
a 1	0.8	0.6	$-\log 0.8 = 0.223$	$-\log 0.6 = 0.511$
a 7	0.2	0.4	$-\log 0.2 = 1.609$	$-\log 0.4 = 0.916$

Certain events cost 0. Rare events cost a lot. Natural logs, so the unit is the **nat**.

Believing Q, a 1 surprises us more than it should (0.511 vs 0.223), a 7 less (0.916 vs 1.609).

KL, STEP 1 OF 3

For each outcome: how much extra surprise does Q cause?

$$\underbrace{-\log Q_i}_{\text{surprise under } Q} - \underbrace{(-\log P_i)}_{\text{surprise under } P} = \log \frac{P_i}{Q_i}$$

outcome	$-\log Q_i$	$-\log P_i$	extra surprise $\log(P_i/Q_i)$
a 1	0.511	0.223	$0.511 - 0.223 = 0.288$
a 7	0.916	1.609	$0.916 - 1.609 = -0.693$

Negative for the 7: Q over-predicts 7s, so a 7 surprises it less than it should.

KL, STEP 2 OF 3

Weight each outcome by how often it really happens

outcome	how often: P_i	extra surprise	weighted: $P_i \log(P_i/Q_i)$
a 1	0.8	0.288	$0.8 \times 0.288 = 0.230$
a 7	0.2	-0.693	$0.2 \times (-0.693) = -0.139$

Real data is 80% 1s, so the 1 row counts four times as much as the 7 row.

KL, STEP 3 OF 3

Add them up: that is the KL divergence

$$\mathrm{KL}(P \,\|\, Q) \;=\; \sum_i P_i \log \frac{P_i}{Q_i}$$

$$= 0.230 + (-0.139) = 0.092 \text{ nats}$$

One row was negative, yet the total can never be: $\mathrm{KL} \geq 0$.

$\mathrm{KL} = 0$ only when $Q = P$ exactly.

KL is the average extra surprise you pay for believing Q when the truth is P.

NOTATION WE WILL USE ALL LECTURE

$\mathbb{E}$ means "average over"

$$\mathbb{E}_{x\sim P}\big[f(x)\big] = \sum_{x} P(x)\, f(x)$$

exact, for a sum over outcomes

$$\mathbb{E}_{z\sim q}\big[f(z)\big] = \int q(z)\, f(z)\, dz$$

exact, for a continuous variable

$$\mathbb{E}_{x\sim P}\big[f(x)\big] \approx \frac{1}{S}\sum_{s=1}^{S} f(x_s), \quad x_s \sim P$$

estimated from samples

KL is an average: $\mathbf{KL}(P\|Q) = \mathbb{E}_{x\sim P}\big[\log \frac{P(x)}{Q(x)}\big]$. Five draws 1, 1, 7, 1, 1 give $\frac{1}{5}(4 \times 0.288 - 0.693) = 0.092$.

NOT A DISTANCE

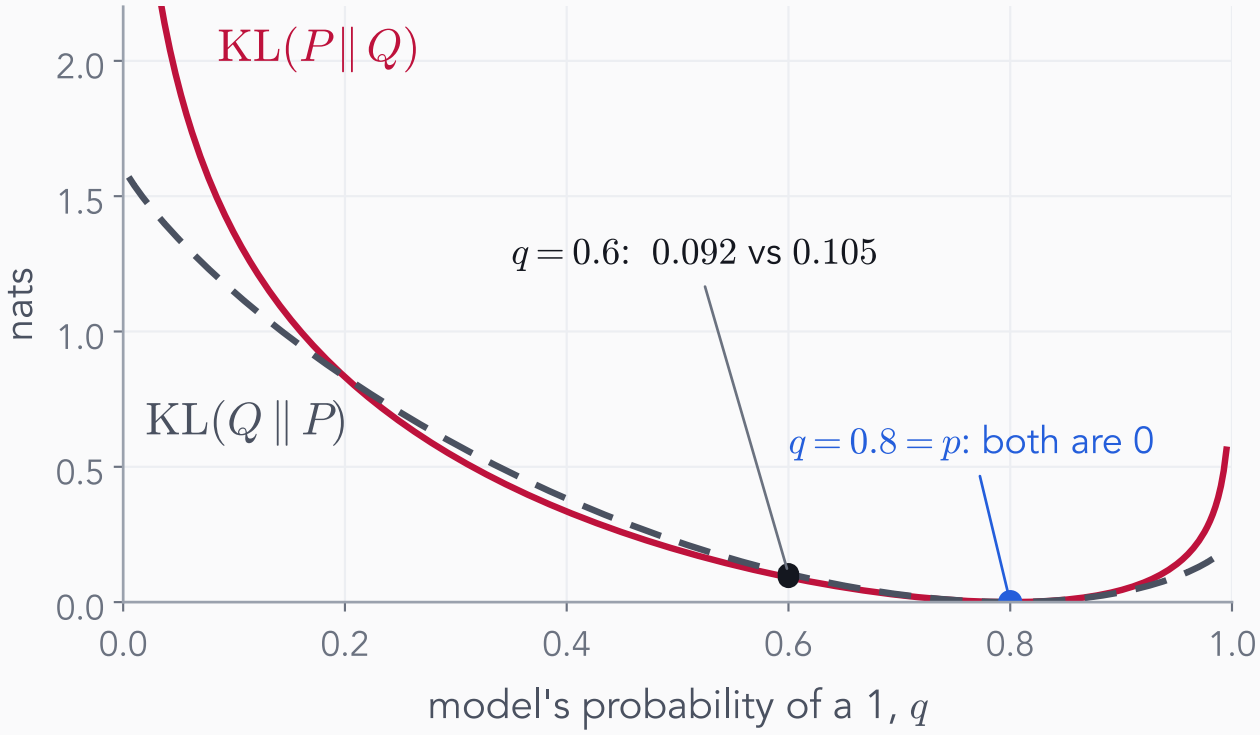
Swap the arguments, get a different number

$$\mathrm{KL}(Q\|P) = 0.6\log\frac{0.6}{0.8} + 0.4\log\frac{0.4}{0.2}$$

$$= -0.173 + 0.277 = 0.105$$

$0.105 \neq 0.092$: KL is not symmetric. "How wrong is Q about P" differs from "how wrong is P about Q".

Push $q \to 0$ and $\mathrm{KL}(P\|Q) \to \infty$. Never give zero probability to something that happens.



YOU HAVE MINIMISED A KL MANY TIMES

Cross-entropy = entropy + KL

$$H(P, Q) = -\sum_i P_i \log Q_i$$

cross-entropy: the loss you train classifiers with

$$= -\sum_i P_i \log P_i + \sum_i P_i \big(\log P_i - \log Q_i \big)$$

add and subtract $\sum_i P_i \log P_i$

$$= \underbrace{H(P)}_{\text{entropy: fixed by the data}} + \mathrm{KL}(P \,\|\, Q)$$

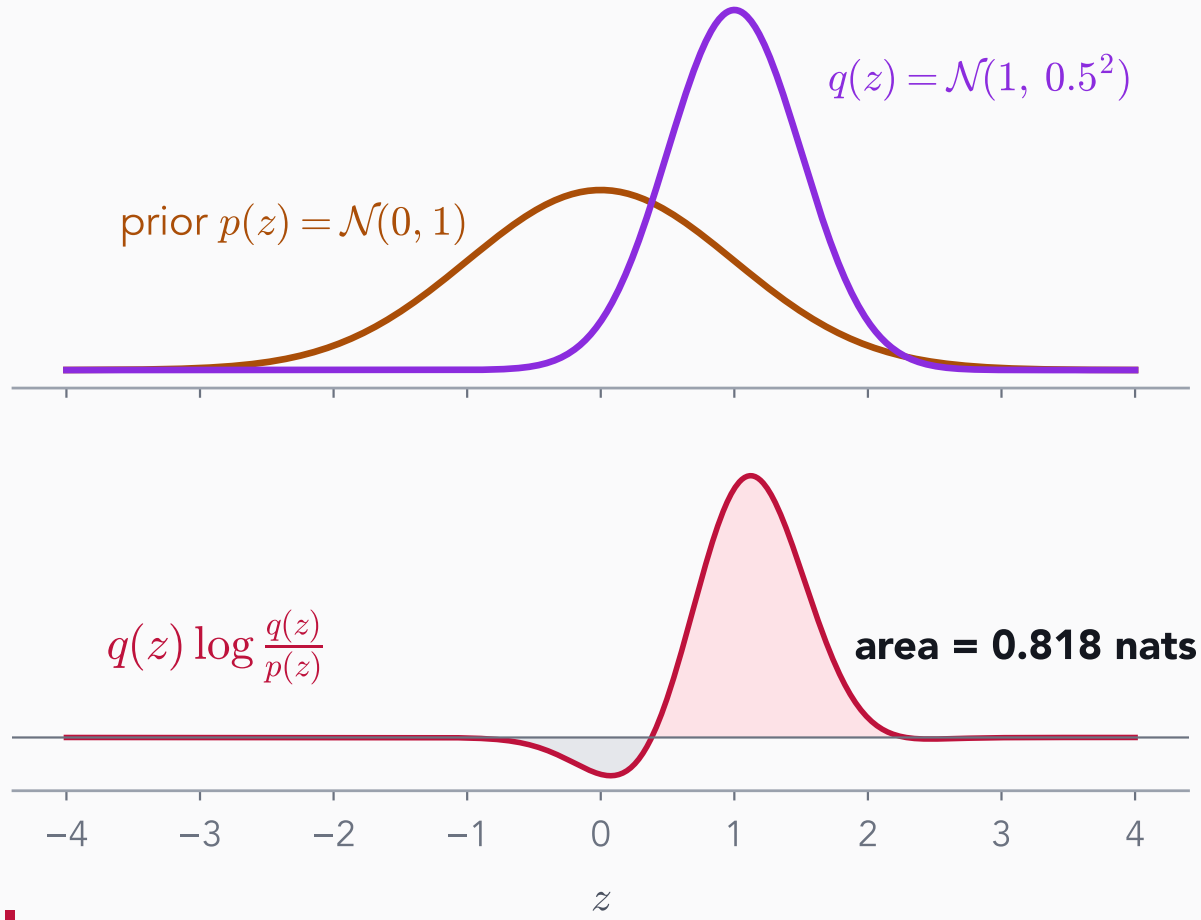
$H(P) = 0.500$
does not depend on the model

0.092
KL

Lowering cross-entropy can only lower the KL part. So does maximum likelihood: it minimises $\mathrm{KL}(\text{data} \,\|\, \text{model})$.

THE VERSION THE VAE NEEDS

KL between two bumps: an area



For continuous variables the sum becomes an integral:

$$\mathrm{KL}(q\|p) = \int q(z)\ \log \frac{q(z)}{p(z)}\, dz$$

Take $q = \mathcal{N}(1, 0.5^2)$, the encoder's cloud, and $p = \mathcal{N}(0, 1)$, the prior.

The shaded area, computed numerically, is **0.818** nats.

Can we get 0.818 without a computer? Yes: for two Gaussians the integral has a formula. Three steps.

GAUSSIAN KL, STEP 1 OF 2

Write out the log of the ratio

$$\log q(z) = -\log \sigma - \tfrac{1}{2}\log 2\pi - \frac{(z-\mu)^2}{2\sigma^2}$$

log of $\mathcal{N}(z; \mu, \sigma^2)$

$$\log p(z) = -\tfrac{1}{2}\log 2\pi - \frac{z^2}{2}$$

log of $\mathcal{N}(z; 0, 1)$

$$\log \frac{q(z)}{p(z)} = -\log \sigma - \frac{(z-\mu)^2}{2\sigma^2} + \frac{z^2}{2}$$

subtract: the $\frac{1}{2}\log 2\pi$ terms cancel

Three terms. Next: average each one over $z \sim q$.

GAUSSIAN KL, STEP 2 OF 2

Average each term over the cloud q

$\mathbb{E}_q\left[(z-\mu)^2\right] = \sigma^2$ (that is the variance)

$\mathbb{E}_q\left[z^2\right] = \mu^2 + \sigma^2$ (mean squared + variance)

$$\mathrm{KL} = -\log\sigma - \frac{\sigma^2}{2\sigma^2} + \frac{\mu^2+\sigma^2}{2} = \frac{1}{2}\left(\mu^2 + \sigma^2 - 1 - \log\sigma^2\right)$$

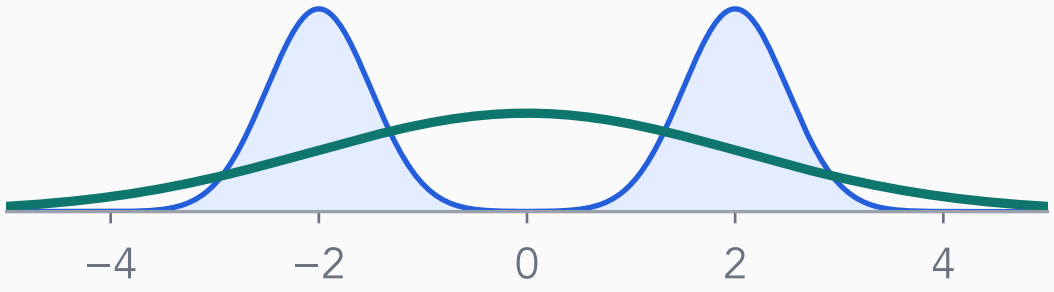
$$\mu = 1,\ \sigma = 0.5: \quad \tfrac{1}{2}\left(1 + 0.25 - 1 + 1.386\right) = 0.818\ \checkmark$$

Moving the mean away costs μ^2; shrinking σ costs $-\log\sigma^2$. For a code with several coordinates, add one such term per coordinate.

BACK TO OUR TWO BUMPS

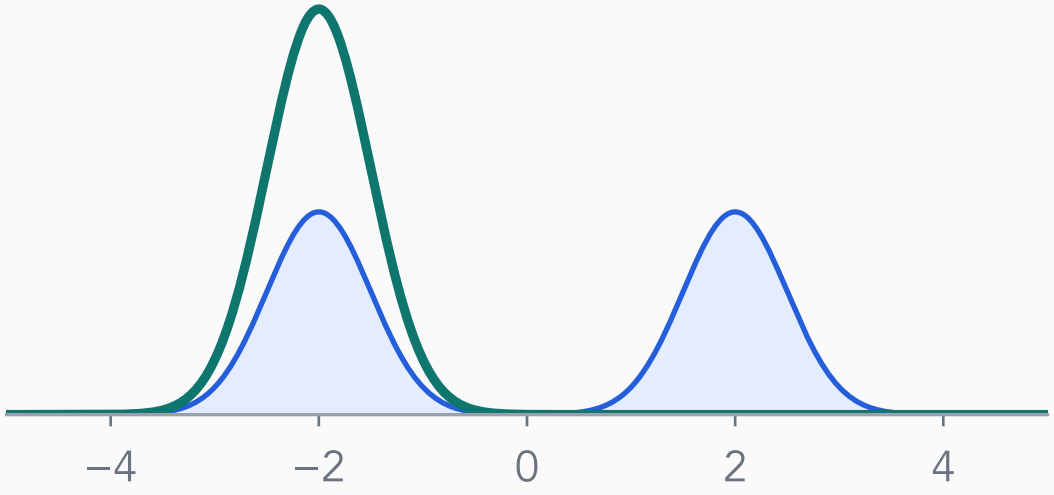
Fit one Gaussian: the KL direction decides what you get

best $q = \mathcal{N}(0.0,\ 2.06^2)$



minimise $\mathrm{KL}(\mathrm{data} \,\|\, q)$: cover everything.
This is the maximum-likelihood fit from before.

best $q = \mathcal{N}(-2.0,\ 0.50^2)$



minimise $\mathrm{KL}(q \,\|\, \mathrm{data})$: never put mass where data is absent, so pick one bump.

Mode-covering vs mode-seeking. Keep this picture: it explains blurry VAEs and mode-collapsing GANs.

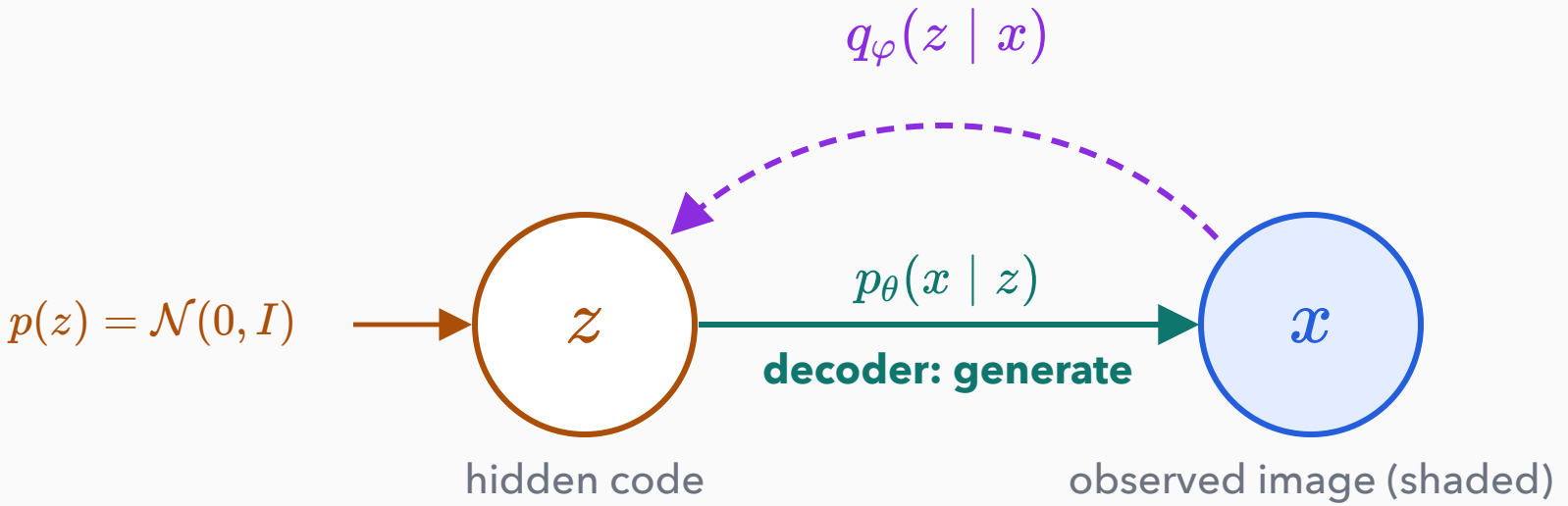
PART 4 · TOOL 2

The ELBO: a loss we can actually compute

Goal: give our 3 high probability, as with the names. Problem: for a latent-variable model that probability is an integral nobody can evaluate. Watch it happen on our 3.

THE MODEL, AS A PICTURE

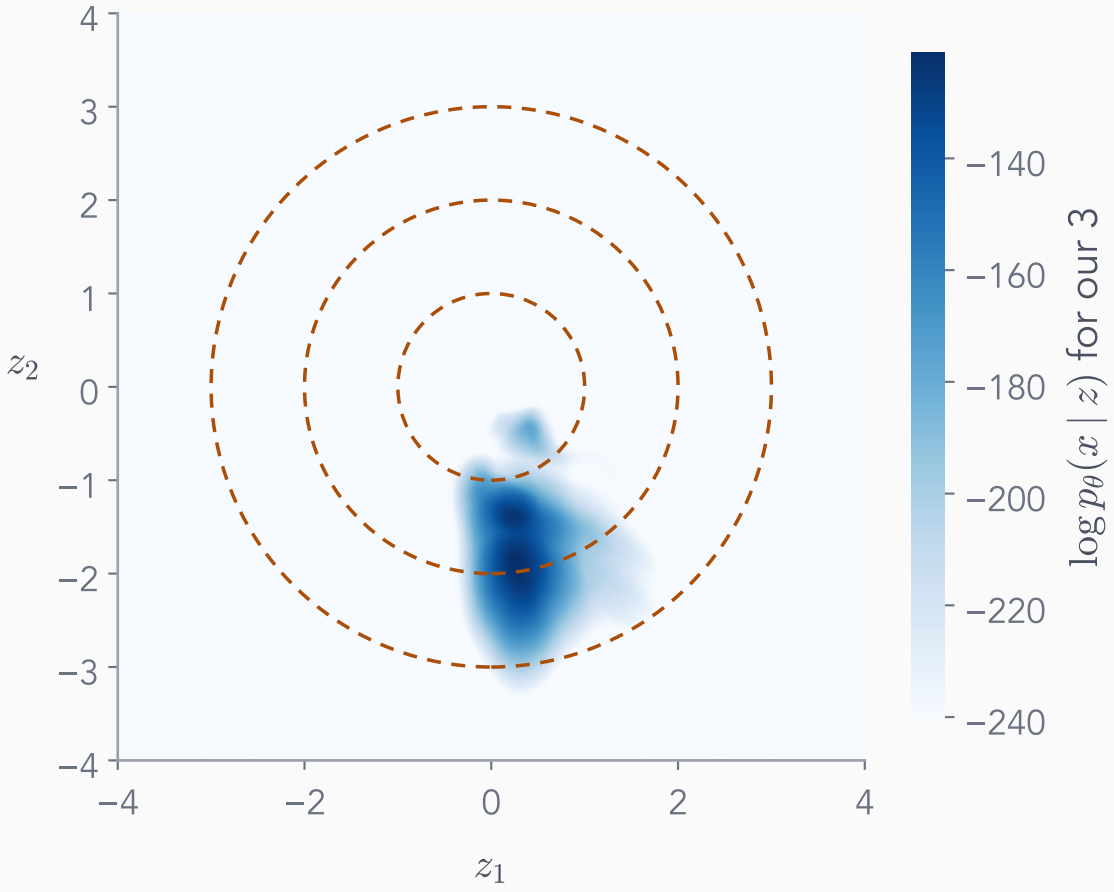
Two arrows: generate down, infer up



$$p_\theta(x, z) = p(z)\, p_\theta(x \mid z)$$

MAXIMUM LIKELIHOOD FOR OUR 3

How probable is our 3 under the model?



With the coin we summed over two values:

$$p(x) = \sum_{z \in \{\mathrm{A},\mathrm{B}\}} p(z)\, p(x \mid z)$$

A code can be any point in the plane, so the sum becomes an integral:

$$p_\theta(x) = \int p(z)\; p_\theta(x \mid z)\, dz$$

2 numbers per code, so we can afford a 901 × 901 grid of decoder calls: $\log p_\theta(x) = -128.7$ nats. With 16 numbers it would be 901^{16} calls.

IDEA: ESTIMATE THE INTEGRAL BY SAMPLING

Random codes almost never hit the right patch

$$p_\theta(x) = \mathbb{E}_{z\sim p(z)}\big[\,p_\theta(x \mid z)\,\big] \approx \frac{1}{S}\sum_{s=1}^{S} p_\theta(x \mid z_s)$$

100,000
random codes $z_s \sim \mathcal{N}(0, I)$

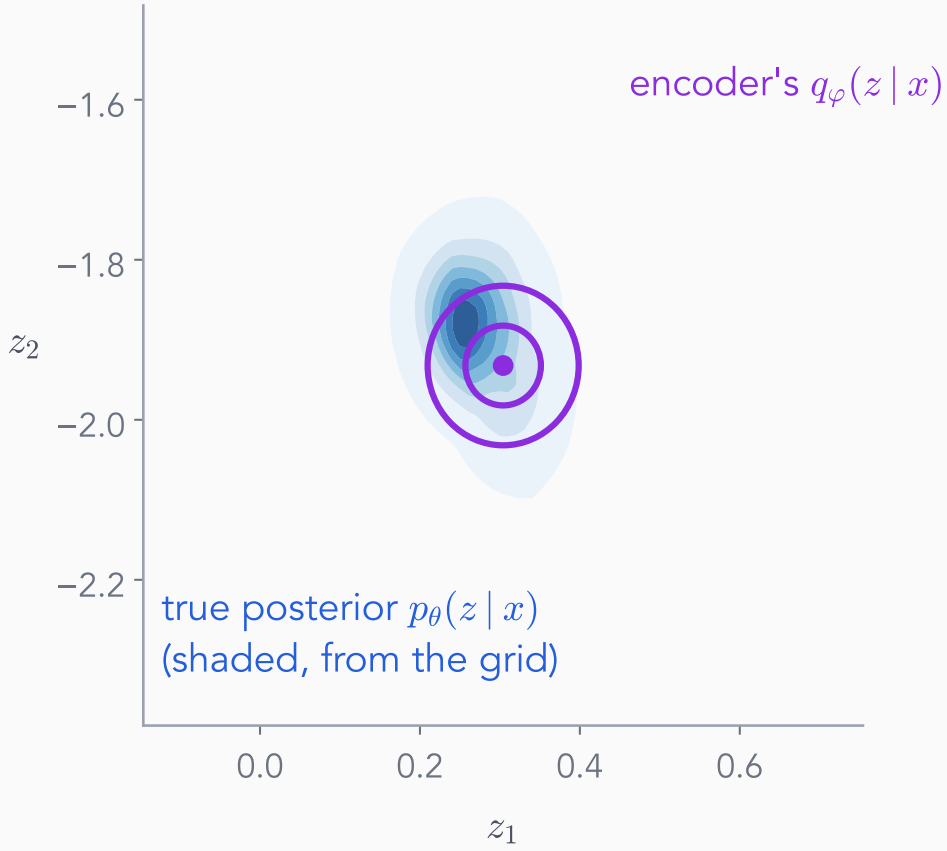
0.07%
land inside our 3's patch

−363.9
typical $\log p(x \mid z)$ of a random code

−121.0
the best code's $\log p(x \mid z)$

> **A typical random code makes our 3 about e^{243} times less probable than the best code. Blind sampling wastes almost every draw.**

Ask the encoder where to look



The patch, normalised, is the true posterior:

$$p_\theta(z \mid x) = \frac{p(z)\, p_\theta(x \mid z)}{p_\theta(x)}$$

It needs $p_\theta(x)$, the impossible integral. So we train a network to guess it:

$$q_\varphi(z \mid x) = \mathcal{N}\big(\mu_\varphi(x),\, \sigma_\varphi(x)^2\big)$$

For our 3: $\mu = (0.3,\ -1.93)$ and $\sigma = (0.047,\ 0.05)$. Close to the truth, not identical.

Start from what we want

$$\log p_\theta(x) = \log \int p_\theta(x, z)\, dz$$

Maximum likelihood wants this number large for our 3, and for every image in the data. The integral over all codes is the part we cannot compute.

We will rewrite this line until it becomes something we can compute and differentiate.

Multiply and divide by the encoder's cloud

$$\log \int p_\theta(x, z)\, dz$$

$$= \log \int q_\varphi(z \mid x)\, \frac{p_\theta(x, z)}{q_\varphi(z \mid x)}\, dz$$

Multiplying and dividing by the same positive number changes nothing. We are free to choose any q; we choose the encoder's Gaussian, which is positive everywhere.

Why bother? Because the next move turns this integral into an average under q, and q knows where the patch is.

Read the integral as an average under q

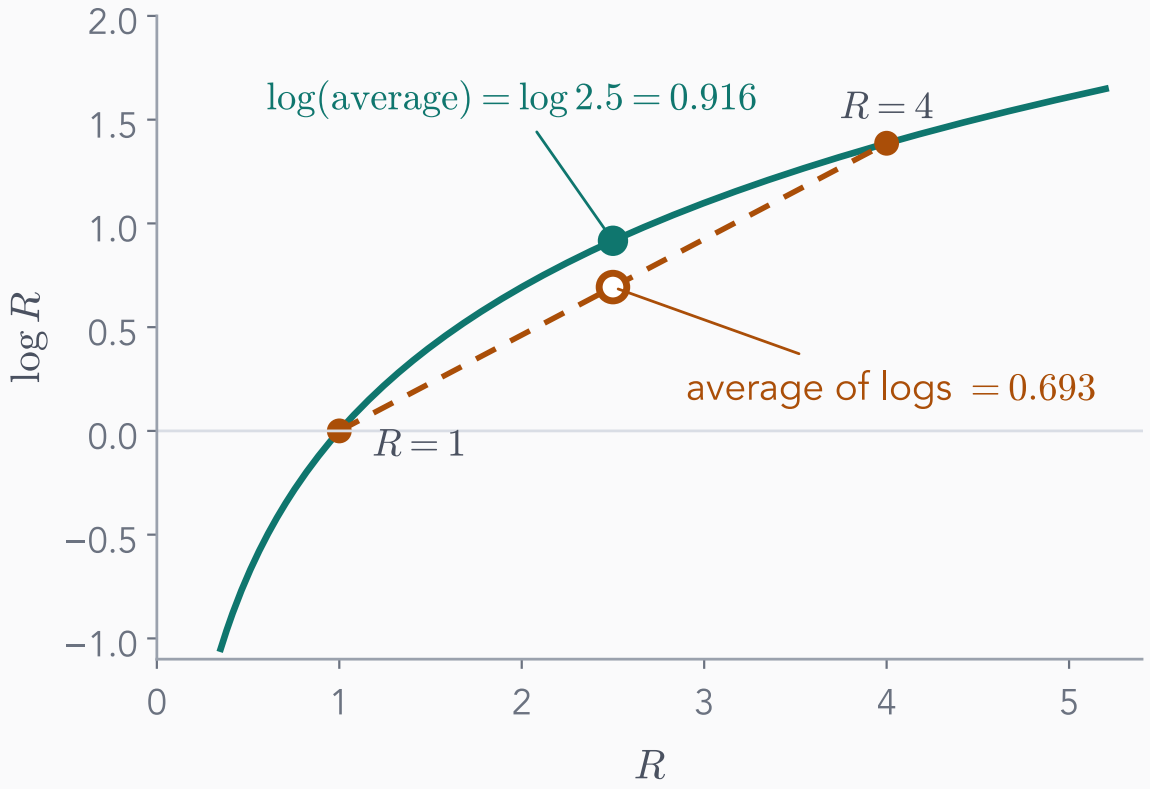
$$\log \int q_\varphi(z \mid x)\, \frac{p_\theta(x, z)}{q_\varphi(z \mid x)}\, dz$$

$$= \log \; \mathbb{E}_{z \sim q_\varphi(z|x)} \left[\frac{p_\theta(x, z)}{q_\varphi(z \mid x)} \right]$$

From the $\mathbb{E}$ slide: $\int q(z)\, f(z)\, dz = \mathbb{E}_{z \sim q}[f(z)]$. Here $f(z)$ is the ratio $p_\theta(x, z)/q_\varphi(z \mid x)$: one number for each code.

Still exact. We now average over codes drawn from the encoder's cloud, right where the patch is.

Jensen's inequality, in one picture



log curves downwards (it is concave), so every chord lies below the curve.

Two values, $R = 1$ and $R = 4$:

$$\underbrace{\log 2.5}_{\log \text{ of the average}} = 0.916 \geq \underbrace{\tfrac{1}{2}(0 + 1.386)}_{\text{average of the logs}} = 0.693$$

$$\log \mathbb{E}[R] \geq \mathbb{E}[\log R]$$

Swap the log and the average: the only inequality

$$\log p_\theta(x) = \log \mathbb{E}_q\left[\frac{p_\theta(x,z)}{q_\varphi(z \mid x)}\right]$$

$$\log p_\theta(x) \geq \mathbb{E}_q\left[\log \frac{p_\theta(x,z)}{q_\varphi(z \mid x)}\right] =: \mathrm{ELBO}$$

Jensen with $R = p_\theta(x,z)/q_\varphi(z \mid x)$. The right side is the **E**vidence **L**ower **BO**und: it can never exceed $\log p_\theta(x)$.

The log is now inside the average. That is what makes the next three moves possible.

Split the log of a product

$$\text{ELBO} = \mathbb{E}_q\left[\log \frac{p_\theta(x \mid z)\, p(z)}{q_\varphi(z \mid x)}\right]$$

$$= \mathbb{E}_q\Big[\log p_\theta(x \mid z) + \log p(z) - \log q_\varphi(z \mid x)\Big]$$

First write $p_\theta(x, z) = p_\theta(x \mid z)\, p(z)$: decoder times prior. Then $\log \frac{ab}{c} = \log a + \log b - \log c$.

The average of a sum is the sum of the averages

$$\mathbb{E}_q\Big[\log p_\theta(x \mid z) + \log p(z) - \log q_\varphi(z \mid x)\Big]$$

$$= \mathbb{E}_q\big[\log p_\theta(x \mid z)\big] + \mathbb{E}_q\big[\log p(z) - \log q_\varphi(z \mid x)\big]$$

$\mathbb{E}[a + b] = \mathbb{E}[a] + \mathbb{E}[b]$. Group the decoder term on its own, and the two terms that only involve z together.

Recognise the second average: it is a KL

$$\mathbb{E}_q\big[\log p(z) - \log q_\varphi(z \mid x)\big]$$

$$= -\,\mathbb{E}_q\left[\log \frac{q_\varphi(z \mid x)}{p(z)}\right] = \boxed{-\mathrm{KL}\big(q_\varphi(z \mid x)\,\|\,p(z)\big)}$$

Flip the sign to flip the fraction. Then use the definition: $\mathrm{KL}(q\|p) = \mathbb{E}_{z\sim q}\big[\log \frac{q(z)}{p(z)}\big]$, the average extra surprise.

And for Gaussians we already have its formula: $\frac{1}{2}\sum_j(\mu_j^2 + \sigma_j^2 - 1 - \log\sigma_j^2)$.

PUT THE PIECES TOGETHER

The ELBO, in symbols and in words

$$\text{ELBO} = \underbrace{\mathbb{E}_{z\sim q}\left[\log p_\theta(x \mid z)\right]}_{\text{rebuild } x \text{ from codes in its cloud}} - \underbrace{\text{KL}\left(q_\varphi(z \mid x) \,\|\, p(z)\right)}_{\text{keep the cloud near } \mathcal{N}(0,I)}$$

First term: the autoencoder's reconstruction score, with the code drawn from the cloud.

Second term: the Gaussian KL we just derived.

The two wishes from the autoencoder slide, derived rather than invented. The loss we minimise is $-\text{ELBO}$.

A second route: start from the true posterior

$$\mathrm{KL}\big(q \,\|\, p_\theta(z \mid x)\big) = \mathbb{E}_q\big[\log q_\varphi(z \mid x) - \log p_\theta(z \mid x)\big]$$

definition of KL, now against the **true posterior**

$$= \mathbb{E}_q\big[\log q_\varphi(z \mid x) - \log p_\theta(x, z)\big] + \log p_\theta(x)$$

Bayes: $\log p_\theta(z \mid x) = \log p_\theta(x, z) - \log p_\theta(x)$, and $\log p_\theta(x)$ has no z in it

$$= -\mathrm{ELBO} + \log p_\theta(x)$$

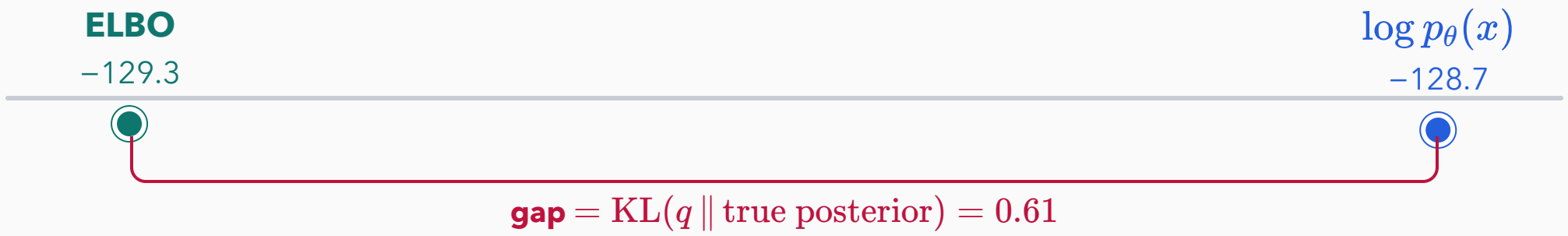
the average is exactly minus the ELBO from move 4

$$\log p_\theta(x) = \mathrm{ELBO} + \mathrm{KL}\big(q_\varphi(z \mid x) \,\|\, p_\theta(z \mid x)\big)$$

The gap is a KL, so it is never negative: a second proof that the ELBO is a lower bound.

THE GAP, MEASURED ON OUR 3

Two different KLs, two different jobs



In the loss: $\mathrm{KL}(q \,\|\, \text{prior}) = 6.97$ nats.
Computable by formula.

In the gap: $\mathrm{KL}(q \,\|\, \text{true posterior}) = 0.61$ nats.
Needs $p_\theta(x)$; we only got it from the grid.

Raising the ELBO raises $\log p_\theta(x)$ or shrinks the gap. Both are what we want.

CHECK THE IDENTITY WITH NUMBERS YOU CAN DO BY HAND

Neat or messy writer? A two-value latent

$z \in \{\text{neat}, \text{messy}\}$, prior $\frac{1}{2}, \frac{1}{2}$. Observation x: a thick stroke.

$$p(x \mid \text{neat}) = 0.8, \quad p(x \mid \text{messy}) = 0.2$$

evidence $p(x) = 0.5$ · true posterior $(0.8,\ 0.2)$

Our guess $q = (q_0,\ 1 - q_0)$:

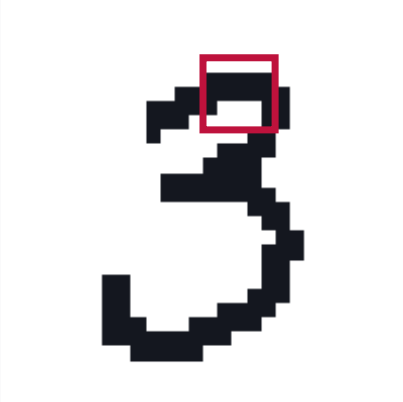
q_0 = `0.60`

quantity	value
$\mathbb{E}_q[\log p(x \mid z)]$	−0.778
$-\,\mathrm{KL}(q \,\|\, \text{prior})$	−0.020
ELBO	**−0.798**
$+\,\mathrm{KL}(q \,\|\, \text{posterior})$	0.105
$\log p(x)$	**−0.693**

Move q: the ELBO and the gap change, their sum never does. At q = posterior (0.8) the gap is 0.

THE RECONSTRUCTION TERM, ZOOMED IN

Each pixel is a coin flip with probability π_i



our 3, one 5 × 5 patch

observed x_i				
0	0	0	0	0
1	1	1	1	1
1	1	1	1	1
1	0	0	0	1
0	0	0	0	1

decoder's π_i				
0.38	0.28	0.13	0.05	0.02
0.89	0.82	0.65	0.41	0.18
0.77	0.79	0.76	0.67	0.40
0.41	0.52	0.67	0.66	0.51
0.22	0.35	0.55	0.60	0.48

$-\log p(x_i \mid z)$				
0.48	0.33	0.14	0.05	0.02
0.11	0.19	0.43	0.89	1.71
0.26	0.24	0.28	0.40	0.91
0.89	0.74	1.12	1.09	0.67
0.25	0.44	0.79	0.93	0.74

this 5 × 5 patch: 14.1 nats

$$-\log p_\theta(x \mid z) = -\sum_{i=1}^{784} \Big[x_i \log \pi_i + (1 - x_i) \log(1 - \pi_i) \Big] = 122.3 \text{ nats}$$

> **Binary cross-entropy is not a design choice: it is the Bernoulli log-likelihood. Gaussian pixels would give squared error.**

FROM THEORY TO THE TRAINING LOSS

The VAE loss for our 3

$$\mathcal{L}(x) = -\text{ELBO} \approx \underbrace{-\log p_\theta(x \mid z)}_{\text{BCE over 784 pixels}} + \underbrace{\frac{1}{2}\sum_{j=1}^{2}\left(\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2\right)}_{\text{KL over 2 coordinates}}, \qquad z \sim q_\varphi(z \mid x)$$

122.3
reconstruction, one draw of z

+ 6.97
KL, exact formula

= 129.2
loss for our 3

> **One problem left: z is a random draw, and we need gradients through it.**